

Tilldelning och jämförelse

Här börjar C-strängar skilja sig från `string`. Varken tilldelning eller jämförelse med C-strängar är speciellt intuitiv, och man behöver för det mesta ta extra funktioner till hjälp.

Tilldelning

Strängar kan inte direkt tilldelas. Eftersom det är frågan om vektorer och således även pekare är det enda som händer att pekarens värde kopieras och båda pekarna refererar ännu till samma minnesområde och sträng. Exempel:

```
#include <iostream>

int main () {
    char Hello1[] = "Hello 1!";
    char * Hello2;

    // skriv ut Hello1
    cout << "Hello1 = '" << Hello1 << "'" << endl << endl;

    // tilldelning
    Hello2 = Hello1;

    // manipulera Hello2
    Hello2 [6] = '2';

    // skriv ut Hello1 och Hello2
    cout << "Hello1 = '" << Hello1 << "'" << endl;
    cout << "Hello2 = '" << Hello2 << "'" << endl;
}
```

Strängen `Hello2` blir inte en kopia, utan den pekar ännu på samma omtåde som `Hello1`. Följdaktligen påverkar manipuleringen av ett tecken i `Hello2` även `Hello1`, vilket visas i utskriften. Där har teckent 1 bytts till 2 i båda strängarna. Vi har gjort en *grund kopia* (shallow copy) av den ursprungliga strängen. I grunda kopior påverkas originalet. Vad vi egentligen för det mesta vill åstadkomma är *djupa kopior* (deep copy) där hela strängens värde kopieras. Detta kan åstadkommas med t.ex följande kod:

```
char Hello1[5] = "Hej!";
char Hello2[5];

// utför kopieringen teckenvis
for ( int Index = 0; Index < 5; Index++ )
    Hello2[Index] = Hello1[Index];
```

Notera att här har vi färdigt allokerat minne för `Hello2`. Man måste alltid försäkra sig om att det finns allokerat minne dit man ämnar kopiera en sträng. Mera om kopiering i [avsnittet Kopiering av strängar](#).

Jämförelse

Jämförelser kan inte direkt utföras med operatoren == eller !=. Inte heller kan man utföra alfabetisk jämförelse med <, <=, >= eller >. Vi måste jämföra teckenvis. Om de jämförda strängarna är lika långa och innehåller exakt samma tecken är de lika. Vi kan skriva en egen jämförelsefunktion på följande sätt:

```
#include <iostream>

bool compare (const char * S1, const char * S2) {
    int Index1 = 0;
    int Index2 = 0;

    // säkerhet
    if ( ! S1 || ! S2 )
        // någondera (eller båda) strängarna odefinierade
        return false;

    while ( S1[Index1] != '\0' && S2[Index2] != '\0' ) {
        // är tecknen ännu lika?
        if ( S1[Index1++] != S2[Index2++] )
            // nope, så strängarna är olika
            return false;
    }

    // strängarna är lika
    return true;
}

int main () {
    char * S1 = "Hej!";
    char * S2 = "Hello world, this is main() calling";
    char * S3 = "Hello world, this is main() calling";

    // jämförelse
    if ( compare ( S1, S2 ) )
        cout << "S1 och S2 är lika" << endl;
    else
        cout << "S1 och S2 är olika" << endl;

    if ( compare ( S2, S3 ) )
        cout << "S2 och S3 är lika" << endl;
    else
        cout << "S2 och S3 är olika" << endl;
}
```

Körning ger följande resultat:

```
% ./CStringCompare1
S1 och S2 är olika
S2 och S3 är lika
%
```

Jämförelsen är alltså ganska trivial till sin natur att implementera, men det finns en färdig funktion som man kan använda, som är en del mångsidigare. Den heter `strcmp()` och ser allmänt ut på följande sätt:

```
#include <string.h>
int strcmp(const char * S1, const char * S2);
```

Den finns definierad i `<string.h>`. Returvärdet är en aning speciellt, och är:

- < 0 om $S1$ är mindre än $S2$.
- $= 0$ om $S1$ är lika med $S2$.
- > 0 om $S1$ är större än $S2$.

Man kan på så sätt enkelt utföra alfabetisk jämförelse mellan två strängar. Funktionen antar att båda strängarna är korrekt terminerade av `'\0'`. Vi kan då skriva om vårt program från ovan på följande sätt:

```
#include <iostream>
#include <string.h>

int main () {
    char * S1 = "Hej!";
    char * S2 = "Hello world, this is main() calling";
    char * S3 = "Hello world, this is main() calling";

    // jämförelse
    if ( strcmp ( S1, S2 ) == 0 )
        cout << "S1 och S2 är lika" << endl;
    else
        cout << "S1 och S2 är olika" << endl;

    if ( strcmp ( S2, S3 ) == 0 )
        cout << "S2 och S3 är lika" << endl;
    else
        cout << "S2 och S3 är olika" << endl;
}
```

Vi är här endast intresserade av om de är lika, inte hur de förhåller sig alfabetiskt till varandra.

En variant på funktionen ovan tar som ett tredje argument antalet tecken som maximalt skall jämföras. Den heter `strncmp()`. Vi kan modifiera programmet ovan en aning:

```
#include <iostream>
#include <string.h>

int main () {
    char * S1 = "Hello world";
    char * S2 = "Hello";

    // jämförelse
    if ( strncmp ( S1, S2, 5 ) == 0 )
        cout << "S1 och S2 är lika" << endl;
    else
        cout << "S1 och S2 är olika" << endl;
}
```

Körning ger nu resultatet:

```
% ./CStringCompare2
S1 och S2 är lika
%
```

Eftersom vi jämförde endast fem tecken av de båda strängarna gav `strncmp()` svaret att de är lika. Med normala `strcmp()` eller ett längre intervall till `strcmp()` hade strängarna varit olika.

[Företgående](#)

Hur C-strängar fungerar

[Hem](#)[Upp](#)[Nästa](#)

Längden av strängar